

Good — here's the updated `Program.cs` with a chat loop that keeps running until you type `exit`, and also keeps track of conversation history so follow-up questions make sense.

```
```csharp
class Program
{
 static async Task Main()
 {
 var embedder = new EmbeddingService();
 var store = new VectorStore();
 var chat = new ChatService();

 // Load and index your document (only done once at startup)
 Console.WriteLine("Loading and indexing document...");
 string documentText = File.ReadAllText("mydocument.txt");
 var chunks = DocumentProcessor.ChunkText(documentText);

 foreach (var chunk in chunks)
 {
 var vector = await embedder.GetEmbedding(chunk);
 store.Add(chunk, vector);
 }

 Console.WriteLine($"Indexed {chunks.Count} chunks. Ready to chat!\n");

 // Keep conversation history across turns
 var conversationHistory = new List<(string role, string content)>();
```

```
while (true)
{
 Console.Write("You: ");
 string question = Console.ReadLine();

 if (string.IsNullOrEmpty(question))
 continue;

 if (question.Trim().ToLower() == "exit")
 {
 Console.WriteLine("Goodbye!");
 break;
 }

 // Find relevant chunks for this question
 var queryVector = await embedder.GetEmbedding(question);
 var relevantChunks = store.Search(queryVector);

 // Ask Llama, passing both document context and chat history
 string answer = await chat.Ask(question, relevantChunks, conversationHistory);

 Console.WriteLine($"Bot: {answer}\n");

 // Save this turn to history
 conversationHistory.Add(("user", question));
```

```

 conversationHistory.Add("assistant", answer));
 }
}
...

```

Update `ChatService.Ask` to accept and use the conversation history:

```

` `` `csharp
class ChatService
{
 private readonly HttpClient _http = new();

 public async Task<string> Ask(string question, List<string> contextChunks, List<(string
role, string content)> history)
 {
 var context = string.Join("\n---\n", contextChunks);

 var systemPrompt = $"Answer ONLY using the following document context. " +
 $"If the answer isn't in the context, say you don't
know.\n\nContext:\n{context}";

 // Build message list: system prompt + prior turns + new question
 var messages = new List<object>
 {
 new { role = "system", content = systemPrompt }
 };
 }
}

```

```

foreach (var (role, content) in history)
{
 messages.Add(new { role, content });
}

messages.Add(new { role = "user", content = question });

var response = await _http.PostAsJsonAsync("http://localhost:11434/api/chat", new
{
 model = "llama3.1",
 messages,
 stream = false
});

var result = await response.Content.ReadFromJsonAsync<ChatResponse>();
return result.message.content;
}
}
...

```

## ## What changed

- The document is indexed **once** at startup, not on every question
- A `while(true)` loop keeps asking for input until you type `exit`
- `conversationHistory` is passed to `ChatService.Ask` each time, so the bot remembers earlier turns (e.g. "what about the second point?" will make sense)

- Each new question still does a fresh retrieval search against your document, so it stays grounded in the source content

## Try it

...

You: What is the refund policy?

Bot: [answer based on document]

You: What about for digital products specifically?

Bot: [answer, understanding "what about" refers to the refund policy]

You: exit

Goodbye!

...

One thing to watch: as history grows, you're sending more tokens each turn, which slows things down and can eventually exceed context length. Want me to add a simple cap (e.g., keep only the last 5-10 turns) so it stays fast in longer sessions?

For reporting from a database, you don't want RAG (that's for unstructured text). Instead, use a **text-to-SQL** pattern: the model generates a SQL query based on your schema, you run it against the database, then feed the results back to the model to summarize into a report.

## Overview

1. Give the model your table schema (not the data itself)
2. User asks a question in plain English
3. Model generates a SQL query
4. You execute that query against your database
5. Feed the results back to the model to write a natural-language report

### ## Step 1: Add a SQL client

```
```bash
```

```
Dotnet add package Microsoft.Data.SqlClient
```

```
Dotnet add package Dapper
```

```
```
```

(Swap for MySQL/PostgreSQL packages if you're not on SQL Server.)

### ## Step 2: Describe your schema to the model

```
```csharp
```

```
Class SchemaProvider
```

```
{
```

```
    Public static string GetSchemaDescription()
```

```
    {
```

```
        Return @"
```

```
Table: Sales
```

```
Columns: Id (int), ProductName (varchar), Quantity (int), Price (decimal), SaleDate  
(datetime), Region (varchar)
```

```
Table: Customers
```

```
Columns: Id (int), Name (varchar), Email (varchar), Region (varchar)
```

```
“,  
}  
}  
...`
```

You can also generate this dynamically by querying `INFORMATION\_SCHEMA.COLUMNS` instead of hardcoding it.

Step 3: Ask Llama to generate SQL

```
` `` `csharp
```

Class SqlGeneratorService

```
{
```

```
    Private readonly HttpClient _http = new();
```

```
    Public async Task<string> GenerateSql(string question, string schema)
```

```
{
```

```
    Var systemPrompt = $"You are a SQL generator. Given a database schema and a  
    question,
```

```
    Output ONLY a valid SQL SELECT query. No explanation, no markdown, just the raw SQL.
```

```
    Never generate INSERT, UPDATE, DELETE, or DROP statements.
```

Schema:

```
{schema}”;
```

```
    Var response = await _http.PostAsJsonAsync(http://localhost:11434/api/chat, new
```

```
{
```

```
    Model = “llama3.1”,
```

```
    Messages = new[]
```

```

    {
        New { role = "system", content = systemPrompt },
        New { role = "user", content = question }
    },
    Stream = false
});

```

```

Var result = await response.Content.ReadFromJsonAsync<ChatResponse>();
Return CleanSql(result.message.content);
}

```

```

Private string CleanSql(string raw)
{
    // Strip markdown fences if the model adds them anyway
    Return raw.Replace("` ` ` sql", "").Replace("` ` `", "").Trim();
}
}
` ` `

```

Step 4: Execute the SQL safely

```
` ` ` csharp
```

```
Using Dapper;
```

```
Using Microsoft.Data.SqlClient;
```

```
Class DatabaseService
```

```
{
```

```
Private readonly string _connectionString =  
"Server=YOUR_SERVER;Database=YOUR_DB;Trusted_Connection=True;TrustServerCertificate=True;";
```

```
Public List<dynamic> ExecuteQuery(string sql)  
{  
    // Safety check: only allow SELECT statements  
    If (!sql.TrimStart().StartsWith("SELECT", StringComparison.OrdinalIgnoreCase))  
        Throw new InvalidOperationException("Only SELECT queries are allowed.");  
  
    Using var connection = new SqlConnection(_connectionString);  
    Var results = connection.Query(sql).ToList();  
    Return results;  
}  
}  
...
```

Step 5: Turn the results into a report

```
```csharp
```

```
Class ReportService
```

```
{
 Private readonly HttpClient _http = new();

 Public async Task<string> GenerateReport(string question, List<dynamic> data)
 {
 String dataJson = System.Text.Json.JsonSerializer.Serialize(data);
```

```
Var systemPrompt = "You are a business analyst. Given raw query results as JSON, " +
 "write a clear, concise report answering the user's question. " +
 "Include key numbers, trends, and a short summary.";
```

```
Var response = await _http.PostAsJsonAsync(http://localhost:11434/api/chat, new
{
 Model = "llama3.1",
 Messages = new[]
 {
 New { role = "system", content = systemPrompt },
 New { role = "user", content = $"Question: {question}\n\nData:\n{dataJson}" }
 },
 Stream = false
});
```

```
Var result = await response.Content.ReadFromJsonAsync<ChatResponse>();
Return result.message.content;
```

```
}
}
...
```

## Step 6: Wire it together

```
```csharp
```

Class Program

```
{  
    Static async Task Main()
```

```

{
    Var sqlGen = new SqlGeneratorService();

    Var db = new DatabaseService();

    Var reportGen = new ReportService();


    String schema = SchemaProvider.GetSchemaDescription();


    While (true)
    {
        Console.Write("Ask a question about your data (or 'exit'): ");

        String question = Console.ReadLine();

        If (question?.ToLower() == "exit") break;


        Try
        {
            String sql = await sqlGen.GenerateSql(question, schema);

            Console.WriteLine($"Generated SQL:\n{sql}\n");


            Var data = db.ExecuteQuery(sql);

            Console.WriteLine($"Rows returned: {data.Count}\n");


            String report = await reportGen.GenerateReport(question, data);

            Console.WriteLine($"Report:\n{report}\n");

        }

        Catch (Exception ex)
        {

```

```
        Console.WriteLine($"Error: {ex.Message}\n");
    }
}
}
}
}
...

```

Example flow

...

Ask: What were total sales by region last month?

Generated SQL:

```
SELECT Region, SUM(Quantity * Price) AS TotalSales
FROM Sales
WHERE SaleDate >= DATEADD(month, -1, GETDATE())
GROUP BY Region
```

Report:

Last month's sales totaled \$142,300 across three regions. The North region led with \$61,200 (43%), followed by South at \$48,100, and West at \$33,000. This represents a 12% increase over the prior month, driven mainly by strong performance in the North.

...

Important safety notes

- ****Never let the model run arbitrary SQL directly against your DB without validation**** — always check it's a `SELECT` only, and ideally use a read-only DB user/connection string for this feature

- Consider a query timeout and row limit (e.g., ``SELECT TOP 1000``) so a bad query can't hang or dump your whole database
- For complex schemas, you may need few-shot examples in the system prompt (2-3 sample question→SQL pairs) so the model generates accurate queries — plain schema descriptions alone often aren't enough for reliable SQL generation

Want me to show how to add few-shot examples to improve SQL accuracy, or how to restrict this to a read-only database user for extra safety?